Préstamos de material del aula

Supuesto de la práctica guiada

Vamos a construir desde una carpeta vacía una aplicación de consola para gestionar el material que el aula presta a sus alumnos. El profesor escribirá el código en pantalla y tú lo irás creando en tu equipo. Pararemos al final de cada etapa para ejecutarlo, explicar las decisiones y resolver dudas.

La aplicación permitirá registrar materiales y alumnos, prestar unidades y registrar devoluciones parciales o completas. También podremos consultar el material disponible, los préstamos pendientes, el historial de cada alumno y un resumen general.

Forma de trabajo

Python 3.13 o superior y VS Code con una terminal abierta en el proyecto. No necesitas instalar paquetes externos ni configurar una base de datos.

Crea una carpeta llamada prestamos_aula. Todos los archivos se escribirán desde cero durante la clase. Conserva las anotaciones de tipo, usa cuatro espacios de sangría y ejecuta los comandos desde la carpeta que contiene main.py.

Datos durante la ejecución

Los datos se guardan en diccionarios en memoria. Los cambios se mantienen mientras el programa sigue abierto y vuelve a mostrar el menú. Al salir y ejecutarlo otra vez se recuperan los datos iniciales: no hay almacenamiento en disco.

El estado se crea una sola vez antes de entrar en el menú. Si lo creamos dentro del bucle, perderemos los cambios entre operaciones.

Qué datos vamos a gestionar

Materiales

Cada material tiene un identificador generado por el programa, un nombre, una cantidad total y una cantidad disponible.

```
"MAT001": {"nombre": "Portátil", "total": 5, "disponibles": 5}
```

Al crear un material solo se piden nombre y cantidad. Todas las unidades empiezan disponibles. El nombre no puede estar vacío y la cantidad debe ser un entero positivo. No se permiten materiales con el mismo nombre, ignorando mayúsculas y espacios sobrantes. Ampliar existencias de un material ya creado queda fuera de esta práctica.

Alumnos

Cada alumno tiene identificador, nombre y activo. Para el alta solo se pide el nombre; el identificador es automático y el alumno comienza activo.

```
"AL001": {"nombre": "Ana", "activo": True}
```

El nombre no puede estar vacío. Dos alumnos pueden llamarse igual: sus identificadores los distinguen. No añadiremos un menú para desactivar alumnos, pero el servicio de préstamos comprobará ese campo.

Préstamos

Cada préstamo relaciona un alumno con un material e indica la cantidad original y las unidades pendientes de devolver.

```
"PRE001": {
    "alumno_id": "AL001", "material_id": "MAT001",
    "cantidad": 2, "pendientes": 2,
}
```

Los identificadores se generan con contadores: MAT001, AL001 y PRE001. Cada alta correcta consume un número; una operación rechazada no lo consume. Un alumno puede tener varios préstamos del mismo material, cada uno con su propio identificador.

Usaremos TypedDict para describir los campos y sus tipos. Son diccionarios normales en ejecución: esta anotación no valida por sí sola sus valores ni nos obliga a crear clases de negocio.

Operaciones y reglas

Registrar un préstamo

Pide el identificador del alumno, el identificador del material y una cantidad entera positiva. El alumno y el material deben existir. El alumno debe estar activo y debe haber unidades suficientes. Si todo es correcto, crea el préstamo y reduce disponibles. El total no cambia.

Antes de modificar los datos hay que comprobar todas las reglas. Si algo falla, muestra un error comprensible y vuelve al menú; no se crea un préstamo ni se cambia el inventario o el contador.

Registrar una devolución

Pide el identificador del préstamo y una cantidad entera positiva. El préstamo debe existir y la cantidad no puede superar sus unidades pendientes. Reduce pendientes y aumenta disponibles en la misma cantidad.

Un préstamo con pendientes igual a cero está completado y se conserva en el historial. No se pueden devolver más unidades de ese préstamo. Su cantidad original no se modifica.

Regla que siempre debe cumplirse

Para cada material: total = disponibles + unidades pendientes de todos sus préstamos. disponibles nunca puede ser negativo ni superar total.

Consultas

- Materiales: identificador, nombre, total, disponibles y prestadas.

- Alumnos: identificador, nombre y estado activo.

- Préstamos pendientes: solo los que tienen pendientes mayor que cero.

- Historial de un alumno: incluye tanto préstamos abiertos como completados. Un alumno existente sin préstamos muestra una lista vacía; un identificador inexistente produce un error.

- Resumen: tipos de material, alumnos registrados, unidades totales, disponibles y prestadas, materiales agotados y número de préstamos pendientes.

El resumen cuenta unidades por separado de los registros. Un préstamo de dos unidades cuenta como un préstamo pendiente y dos unidades prestadas.

Estructura de la aplicación

```
prestamos_aula/
  main.py
  aula/
    __init__.py
    dominio/
      __init__.py
      modelos.py
      errores.py
    datos/
      __init__.py
      memoria.py
    servicios/
      __init__.py
      materiales.py
      alumnos.py
      prestamos.py
      resumen.py
      decoradores.py
    interfaz/
      __init__.py
      consola.py
```

Responsabilidades

dominio describe los datos y los errores previstos. datos construye el estado inicial. servicios aplica las reglas y las consultas. interfaz pide y convierte entradas y muestra resultados. main.py crea el estado y arranca el menú.

Los servicios reciben el estado como argumento. No piden datos con input y no crean una copia del almacén en cada operación. Empezaremos con importaciones absolutas a módulos concretos. Al final reexportaremos los nombres públicos en los __init__.py para importar entre capas sin indicar el módulo. Los imports internos de una misma capa seguirán apuntando al módulo concreto para evitar ciclos.

Punto de partida

Materiales iniciales: MAT001 Portátil con 5 unidades y MAT002 Proyector con 2 unidades, todas disponibles. Alumnos iniciales: AL001 Ana y AL002 Luis, activos. No hay préstamos. El próximo material será MAT003, el próximo alumno AL003 y el próximo préstamo PRE001.

Primero usaremos main.py para pequeñas demostraciones. Cuando incorporemos el menú, crearemos un único estado por ejecución. Reiniciar durante el desarrollo también reinicia los datos: es el comportamiento esperado.

Ruta de desarrollo del alumno

Sigue estos pasos en orden. Al terminar cada uno, comprueba el resultado antes de continuar. Si la clase se detiene, anota el último paso que funciona y retoma el siguiente. Las operaciones y reglas de las páginas anteriores son el contrato de la aplicación.

Paso 1 Preparar paquetes y estado inicial

Crea la estructura de carpetas y sus __init__.py, inicialmente vacíos. En dominio/modelos.py describe los tipos Material, Alumno, Prestamo y EstadoAula. El estado agrupa los tres diccionarios y los contadores de identificadores. En datos/memoria.py escribe crear_estado_inicial. Desde main.py llama a esa función y muestra los datos iniciales.

Comprueba: aparecen los dos materiales y los dos alumnos indicados en el supuesto, sin préstamos. Dos llamadas a crear_estado_inicial deben producir estados independientes. Mantén los imports dirigidos a módulos concretos hasta el paso 11.

Paso 2 Crear errores y altas de material

En dominio/errores.py define ErrorAula y sus errores derivados: nombre inválido, cantidad inválida, material duplicado, entidad inexistente, alumno inactivo, stock insuficiente y devolución excesiva. Los errores de stock y devolución deben conservar las cantidades implicadas como atributos.

En servicios/materiales.py añade listar_materiales, obtener_material y crear_material. Limpia el nombre, valida antes de modificar y genera el identificador con el contador. Captura los errores previstos en las demostraciones temporales de main.py.

Comprueba: crear Tablet con 8 unidades genera MAT003, total 8 y disponibles 8. Repetir el nombre con otras mayúsculas debe fallar sin cambiar datos ni consumir otro identificador.

Paso 3 Crear altas y consulta de alumnos

En servicios/alumnos.py escribe listar_alumnos, obtener_alumno y crear_alumno. El alta necesita un nombre no vacío, un identificador nuevo y activo igual a verdadero. Permite nombres repetidos.

Comprueba: dos altas llamadas Marta obtienen AL003 y AL004. Un nombre vacío se rechaza sin consumir un número.

Paso 4 Registrar préstamos

En servicios/prestamos.py escribe prestar_material. Recibe el estado, el alumno, el material y la cantidad. Reutiliza las búsquedas anteriores, valida todas las reglas y después crea el préstamo, reduce disponibles e incrementa el contador. Devuelve el identificador generado.

Comprueba: prestar 2 portátiles a Ana genera PRE001 y deja 3 disponibles. Pedir después 4 debe fallar y mantener los 3. Haz estas comprobaciones en la misma ejecución.

Paso 5 Registrar devoluciones

En el mismo módulo añade devolver_material. Busca el préstamo y comprueba que la cantidad sea positiva y no supere pendientes. Después actualiza pendientes y disponibles. Devuelve las unidades pendientes; no borres el registro ni cambies la cantidad original.

Comprueba: devolver una unidad del préstamo anterior deja 1 pendiente y 4 disponibles. Otra devolución de una unidad lo completa y recupera las 5 disponibles. Una devolución adicional se rechaza sin cambiar nada.

Ruta de desarrollo del alumno continuación

Paso 6 Consultar préstamos y preparar el resumen

Añade consultar_prestamos a servicios/prestamos.py. Permite filtrar por alumno y usa solo_pendientes como opción que se pasa por nombre. El historial debe permitir incluir registros completados. En servicios/resumen.py escribe sumar_cantidades con argumentos posicionales variables y obtener_resumen con los indicadores del supuesto.

Comprueba: un préstamo completado desaparece de pendientes, pero sigue en el historial de su alumno. Tras devolver todo, hay 7 unidades disponibles y ninguna prestada en el estado inicial sin altas adicionales.

Paso 7 Separar entrada y presentación

En interfaz/consola.py crea leer_entero, mostrar_materiales, mostrar_alumnos y mostrar_prestamos. Convierte el texto numérico y transforma ValueError en CantidadInvalidaError conservando la causa. Presenta los resultados de los servicios sin trasladar allí las reglas de negocio.

Comprueba: los listados se entienden y una consulta vacía muestra un mensaje. Una entrada como dos produce el error de conversión previsto, no un entero. Las cantidades positivas se seguirán validando en los servicios.

Paso 8 Construir el menú y mantener la sesión

Añade mostrar_menu y ejecutar_menu con las opciones del supuesto. Repite el menú hasta salir, normaliza los identificadores introducidos y captura ErrorAula dentro del bucle. Sustituye las demostraciones de main.py por el arranque definitivo: crear el estado una sola vez y pasarlo al menú.

Comprueba: puedes crear material y un alumno, prestarles unidades y devolverlas sin reiniciar. Una opción desconocida o una cantidad no numérica no cierra el programa. Antes de este paso, los cambios solo duraban lo que duraba cada demostración de main.py.

Ruta de desarrollo del alumno cierre

Paso 9 Añadir el decorador de registro

En servicios/decoradores.py crea formatear_evento con metadatos nombrados variables y registrar_operacion. Su envoltorio debe registrar INICIO, llamar a la función, registrar OK y devolver su resultado. Ante ErrorAula, debe registrar ERROR y volver a lanzar la misma excepción. Conserva nombre y documentación con wraps y mantén el tipado de parámetros y retornos.

Solo cuando el módulo esté terminado, aplica el decorador a crear_material, crear_alumno, prestar_material y devolver_material. Comprueba que las consultas no muestran esos registros y que un rechazo no imprime OK ni modifica el estado. El menú muestra el motivo del error una sola vez.

Paso 10 Comprobar el conjunto

Ejecuta el recorrido y los casos de error de la sección Comprobación final. Revisa especialmente los límites: prestar todas las unidades disponibles, devolver exactamente lo pendiente y rechazar una unidad adicional. Confirma la relación entre total, disponibles y pendientes.

Si dispones de las pruebas de referencia del profesor, ejecútalas con unittest. Si aún no las tienes, utiliza el recorrido manual y anota los casos que fallan para corregirlos antes de continuar. No cambies las reglas para que una prueba deje de fallar.

Paso 11 Definir las interfaces públicas

Completa los __init__.py de dominio, datos, servicios e interfaz con los nombres públicos y __all__. Expón tipos y errores, crear_estado_inicial, las operaciones que utiliza la consola y ejecutar_menu, respectivamente. Simplifica los imports entre capas; dentro de una misma capa conserva los imports a módulos concretos para evitar ciclos.

No crees el estado ni arranques el menú al importar un paquete. Comprueba otra vez el recorrido final y las pruebas disponibles: esta reorganización no debe cambiar los resultados ni introducir errores de importación.

Menú y registro de operaciones

Menú final

```
1. Mostrar materiales
2. Crear material
3. Mostrar alumnos
4. Crear alumno
5. Registrar préstamo
6. Registrar devolución
7. Consultar préstamos pendientes
8. Consultar historial de un alumno
9. Mostrar resumen
0. Salir
```

Si la cantidad no se puede convertir con int, se mostrará un error y el menú continuará. Los identificadores introducidos por consola se limpiarán y pasarán a mayúsculas. Una opción de menú desconocida no cerrará la aplicación.

Decorador de registro

Cuando las operaciones funcionen, construiremos registrar_operacion y lo aplicaremos a las altas, los préstamos y las devoluciones. Mostrará INICIO y, después, OK o ERROR. Si la operación falla, el decorador vuelve a lanzar el mismo error para que el menú muestre el motivo una sola vez.

El envoltorio debe reenviar todos los argumentos, conservar el resultado y usar wraps. Por eso no basta con escribir mensajes: debe seguir devolviendo el identificador de un alta o el número de unidades pendientes de una devolución.

Practicaremos *args y **kwargs para reenviar llamadas y crear mensajes de registro. La consulta de préstamos tendrá un parámetro keyword-only, solo_pendientes. Para conservar los tipos de funciones con firmas y retornos diferentes, el profesor explicará la notación [**P, R] de Python 3.13. No usaremos Protocol ni un framework.

Las consultas no se registran con este decorador. No añadiremos reintentos a operaciones que modifican préstamos: no hay red ni un fallo transitorio que los justifique.

Comprobación final

Recorrido que debe funcionar

1. Ejecuta python main.py y crea Tablet con cantidad 8. Debe obtener MAT003 con total 8 y disponibles 8.

2. Crea a Marta. Debe obtener AL003 y estar activa.

3. Presta 2 unidades de MAT003 a AL003. Se crea PRE001 y quedan 6 disponibles.

4. Devuelve 1 unidad de PRE001. Quedan 7 disponibles y 1 pendiente.

5. Devuelve otra unidad. Vuelven a estar las 8 disponibles y el préstamo queda completado.

6. Los pendientes ya no incluyen PRE001, pero el historial de AL003 sí lo conserva.

Prueba también los errores

- Crear un nombre vacío, cantidad cero o un material duplicado.

- Prestar a un alumno inexistente o pedir más unidades de las disponibles.

- Devolver más unidades de las pendientes o devolver otra vez un préstamo completado.

- Escribir dos en una entrada numérica y comprobar que el menú continúa.

Antes y después de cada rechazo, revisa los datos: no deben cambiar.

Cómo ejecutar

Desde la carpeta que contiene main.py:

```
python --version
python main.py
```

En la última etapa el profesor facilitará las pruebas de referencia. Podrás copiarlas a tests y ejecutarlas sin instalar herramientas:

```
python -m unittest discover -s tests -v
```

Qué conservar al terminar

Tu proyecto completo, con los paquetes, código tipado y las operaciones funcionando. Debes poder explicar dónde vive el estado, qué validaciones se hacen antes de cambiarlo y cómo el decorador conserva el resultado y los errores.

Si terminas antes, propón cómo ampliar unidades de un material o desactivar un alumno. Estas mejoras no forman parte de la versión que construiremos todos juntos.